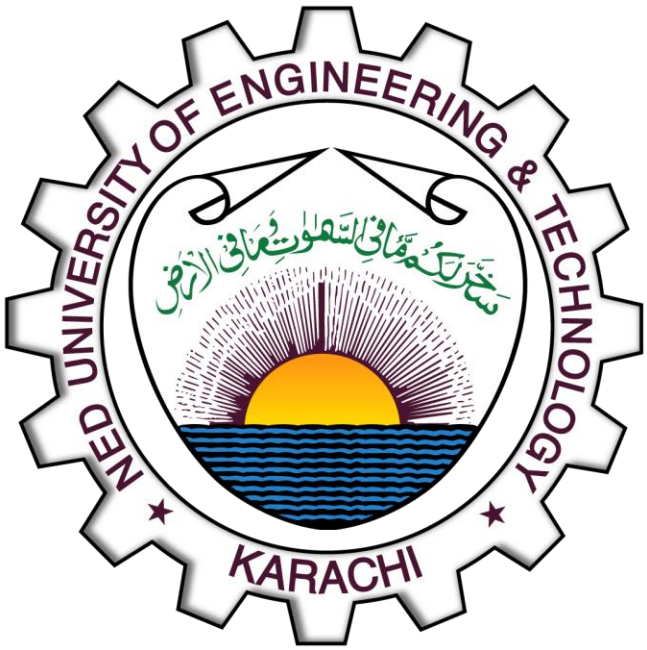
DATA STRUCTURE ALGORITHMS & APPLICATIONS

[CT-159]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: SSCS-D

LAB : #04

QUESTION#1:

CODE:

```
#include <iostream>
#include <stack>
using namespace std;

bool palindrome(const string& s) {
    stack<char> check;
    for (char c : s)
        check.push(c);
    for (char c : s) {
        if (c != check.top()) return false;
        check.pop();
    }
    return true;
}

int main() {
    string s;
    cout << "Enter a string: ";
    getline(cin, s);
    cout << "Is it a palindrome? " << (palindrome(s) ? "Yes" : "No") << endl;
    return 0;
}
```

OUTPUT:

```
Enter a string: 1221
Is it a palindrome? Yes
```

QUESTION#2:

CODE:

```
#include <iostream>
#include <stack>
using namespace std;

string stringProcessing(const string &s)
{
    string result;
    for (char c : s)
        if (c == '#')
        {
            if (!result.empty())
                result.pop_back();
        }
        else
            result.push_back(c);
    return result;
}

bool equalStrings(const string &s, const string &t)
{
    return stringProcessing(s) == stringProcessing(t);
}

int main()
{
    string s, t;
    cout << "Enter the strings:\n";
    getline(cin, s);
    getline(cin, t);
    cout << "The strings are " << (equalStrings(s, t) ? "Equal" : "Unequal") <<
endl;
    return 0;
}
```

OUTPUT:

```
Enter the strings:
ab#c
ad#c
The strings are Equal
```

QUESTION#3:

CODE:

```
#include <bits/stdc++.h>
using namespace std;

void backtrackPermute(vector<int> &arr, size_t index, vector<vector<int>>
&result)
{
    if (index == arr.size())
    {
        result.push_back(arr);
        return;
    }
    for (size_t i = index; i < arr.size(); i++)
    {
        swap(arr[index], arr[i]);
        backtrackPermute(arr, index + 1, result);
        swap(arr[index], arr[i]);
    }
}

vector<vector<int>> permute(vector<int> arr)
{
    vector<vector<int>> result;
    backtrackPermute(arr, 0, result);
    return result;
}

int main()
{
    vector<int> arr;
    int input;
    while (true)
    {
        cin >> input;
        if (input)
            arr.push_back(input);
        else
            break;
    }
    cout << "Given Array: ";
    for (size_t i = 0; i < arr.size(); i++)
```

```
    {
        cout << arr[i] << " ";
    }
    vector<vector<int>> res(permute(arr));
    cout << "\nTotal permutation of the given array:\n";
    for (auto subarr : res)
    {
        for (auto element : subarr)
            cout << element << " ";
        cout << endl;
    }
    return 0;
}
```

OUTPUT:

```
1
2
3
0
Given Array: 1 2 3
Total permutation of the given array:
1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2
```

QUESTION#4:

CODE:

```
#include <iostream>
#include <vector>
using namespace std;

bool dfs(const vector<vector<char>> &grid, vector<vector<bool>> &visited, size_t
r, size_t c, size_t index, const string &word)
{
    if (index == word.size())
        return true;
    if (r < 0 || r >= grid.size() || c < 0 || c >= grid[0].size() || grid[r][c]
!= word[index] || visited[r][c])
        return false;
    visited[r][c] = true;
    int neighbours[][2] = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    bool found = false;
    for (size_t i = 0; i < 4; i++)
        found |= dfs(grid, visited, r + neighbours[i][0], c + neighbours[i][1],
index + 1, word);
    visited[r][c] = false;
    return found;
}

bool exist(const vector<vector<char>> &grid, const string &word)
{
    size_t rows = grid.size();
    size_t cols = grid[0].size();
    vector<vector<bool>> visited(rows, vector<bool>(cols, false));
    for (size_t r = 0; r < rows; r++)
    {
        for (size_t c = 0; c < cols; c++)
            if (grid[r][c] == word[0] && dfs(grid, visited, r, c, 0, word))
                return true;
    }
    return false;
}

int main()
{
    size_t m, n;
    cout << "Enter the size of the grid:\n";
```

```
    cout << "Rows: ";
    cin >> m;
    cout << "Cols: ";
    cin >> n;
    vector<vector<char>> grid(m, vector<char>(n, '.'));
    for (size_t i = 0; i < m; i++)
        for (size_t j = 0; j < n; j++)
            cin >> grid[i][j];
    cin.ignore();
    string word;
    cout << "Enter the word to find: ";
    getline(cin, word);
    cout << "\nThe Grid:\n";
    for (size_t i = 0; i < m; i++)
    {
        for (size_t j = 0; j < n; j++)
            cout << grid[i][j] << " ";
        cout << endl;
    }
    cout << endl
         << "Does the word exist? " << (exist(grid, word) ? "Yes" : "No") <<
endl;
    return 0;
}
```

OUTPUT:

```
Enter the size of the grid:
Rows: 3
Cols: 4
A B C E
S F C S
A D E E
Enter the word to find: ABCCE

The Grid:
A B C E
S F C S
A D E E

Does the word exist? Yes
```